



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

“Laboratorio N°01 – Revisión de Código”

Curso: Construcción de Software II

Docente: Mag. Ricardo Eduardo Valcarcel Alvarado

Integrantes:

- Cuadros Garcia, Mirian (2021071083)
- Llantay Machaca, Marjiory Grace (2020068951)

Proyecto de Implementación

**Aplicativo de Monitoreo en tiempo real basado en Internet de las
Cosas para el control del habita apicola en el
museo de Pampachacra, 2025**

Laboratorio 01 – Revisión de Código

Presentado por:

Cuadros Garcia, Mirian
Llantay Machaca, Marjiory Grace

08/08/2025

CONTROL DE VERSIONES							
Versión	Hecha por			Revisada por	Aprobada por	Fecha	Motivo
1.1	Llantay Machaca, Marjory Grace Cuadros Garcia, Mirian			Ricardo Valcarcel Alvarado	Ricardo Valcarcel Alvarado	08/08/2025	Versión 1



ÍNDICE

Introducción	3
Guía de Laboratorio N° 01 – REVISION DE CODIGO.....	4
1. Información sobre el evento práctico.....	4
1.1. Objetivos.....	4
1.2. Equipos, materiales, programas y recursos.....	4
1.3. Seguridad	4
1. Desarrollo de la Práctica	4
2. Conclusiones.....	5





Introducción

En el desarrollo de software, especialmente en proyectos colaborativos, es fundamental contar con lineamientos claros de estilo y buenas prácticas de programación que aseguren la calidad, legibilidad y mantenibilidad del código. Estos estándares, que pueden variar según el equipo o el proyecto, establecen detalles como la organización de las clases y funciones, la estructura de la indentación, así como el uso correcto de llaves, paréntesis y nombres de variables. Bajo esta premisa, el Laboratorio N°01 – Revisión de Código tiene como finalidad analizar, depurar y optimizar el código implementado en los módulos asignados, identificando duplicidades, fallos, variables mal definidas y valores poco eficientes. El objetivo es refinar el trabajo realizado para que, al momento de la entrega final, el producto cumpla con criterios aceptables de calidad, seguridad y rendimiento.

Guía de Laboratorio N° 01 – REVISION DE CODIGO

1. Información sobre el evento práctico

1.1. Objetivos

- Realizar la revisión del código contemplando buenas prácticas y estándares de construcción de software.

1.2. Equipos, materiales, programas y recursos

- Software Ofimático
- Software requerido para su aplicación
- Aplicación diseñada e implementada en el Curso de Construcción de Software I
- Formatos de despliegue para la documentación del Proyecto de Software en proceso de implementación

1.3. Seguridad

- Protocolo de Seguridad para laboratorios y talleres
- Protocolo ante la presencialidad en Laboratorios



1. Desarrollo de la Práctica

El equipo formado por nosotras enfocado en Aplicativo de Monitoreo en tiempo real basado en Internet de las Cosas para el control del habita apícola en el museo de Pampachacra, 2025.

- Tiene como propósito cumplir con los siguientes propósitos:
 - Mejorar el código implementado
 - Mejorar el aplicativo
- Luego de tener el contexto que debemos mejorar en el proyecto tendremos como objetivos evidenciar dentro de nuestro código lo siguiente:
 - Duplicidad del código
 - Revelar fallos
 - Corregir variables
 - Optimizar los valores

Requerimientos

✓ RF- 001 Autenticación de Usuario

Permitir el inicio de sesión seguro y acceso al sistema, diferenciando entre superusuario y operadores.

```
1  import 'package:firebase_auth/firebase_auth.dart';
2  import 'package:cloud_firestore/cloud_firestore.dart';
3
4  class AuthController {
5    final FirebaseAuth _auth = FirebaseAuth.instance;
6    final FirebaseFirestore _db = FirebaseFirestore.instance;
7
8    Future<User>> login(String email, String password) async {
9      final result = await _auth.signInWithEmailAndPassword(
10        email: email,
11        password: password,
12      );
13      return result.user;
14    }
15
16    Future<void> logout() async {
17      await _auth.signOut();
18    }
19
20    Future<User>> register(
21      String email,
22      String password,
23      String nombre,
24      String rol,
25    ) async {
26      final result = await _auth.createUserWithEmailAndPassword(
27        email: email,
28        password: password,
29      );
30      final uid = result.user!.uid;
31
32      await _db.collection('usuarios').doc(uid).set({
33        'nombre': nombre,
34        'email': email,
35        'rol': rol,
36      });
37
38      return result.user;
39    }
40
41    Stream<User>> get userChanges => _auth.authStateChanges();
42  }
```

En el código observamos la falta de comentarios

DESCRIPCIÓN DEL CÓDIGO

El código implementa un controlador de autenticación de usuarios en Flutter usando Firebase Authentication y Cloud Firestore.

Está estructurado en la clase AuthController, que se encarga de:

1. Conexión con Firebase
 - ✓ FirebaseAuth para autenticación de usuarios.
 - ✓ Firestore para guardar datos adicionales de cada usuario.
2. Método login
 - ✓ Permite que un usuario inicie sesión con correo y contraseña (signInWithEmailAndPassword).
 - ✓ Retorna el objeto User en caso de éxito.
3. Método logout
 - ✓ Cierra la sesión del usuario actual (signOut).
4. Método register
 - ✓ Crea un nuevo usuario con correo y contraseña (createUserWithEmailAndPassword).
 - ✓ Guarda información adicional (nombre, email y rol) en la colección usuarios de Firestore.
 - ✓ Retorna el objeto User creado.
5. Propiedad userChanges
 - ✓ Devuelve un Stream<User?> para detectar cambios en el estado de autenticación en tiempo real (authStateChanges).

Propósito dentro del proyecto

En el contexto del Aplicativo de Monitoreo en Tiempo Real basado en IoT para el Control del Hábitat Apícola:

- ✓ Este módulo asegura que solo usuarios autorizados (administradores) puedan acceder a los datos de sensores (CO₂, sonido, peso)
- ✓ El control de roles (rol) permitirá diferenciar permisos para visualización, configuración o mantenimiento.

Mejoras realizadas

- ✓ Se realizola implementacion de los comentarios respectivos por cada linea de codigo explicando brevemente.

```
1 import 'package:firebase_auth/firebase_auth.dart'; // Librería para autenticación con Firebase
2 import 'package:cloud_firestore/cloud_firestore.dart'; // Librería para la base de datos Firestore
3
4 // Controlador de autenticación que gestiona inicio de sesión, registro y cierre de sesión
5 class AuthController {
6   // Instancia de Firebase Authentication
7   final FirebaseAuth _auth = FirebaseAuth.instance;
8   // Instancia de Firestore para almacenar y recuperar datos de usuarios
9   final FirebaseFirestore _db = FirebaseFirestore.instance;
10
11   // Método para iniciar sesión con correo y contraseña
12   Future<User?> login(String email, String password) async {
13     final result = await _auth.signInWithEmailAndPassword(
14       email: email,
15       password: password,
16     );
17     return result.user; // Devuelve el usuario autenticado
18   }
19
20   // Método para cerrar sesión
21   Future<void> logout() async {
22     await _auth.signOut();
23   }
24
25   // Método para registrar un nuevo usuario con datos adicionales en Firestore
26   Future<User?> register(
27     String email,
28     String password,
29     String nombre,
30     String rol,
31   ) async {
32     // Crear usuario en Firebase Authentication
33     final result = await _auth.createUserWithEmailAndPassword(
34       email: email,
35       password: password,
36     );
37     final uid = result.user!.uid; // Obtener el identificador único del usuario
38
39     // Guardar datos adicionales del usuario en la colección "usuarios"
40     await _db.collection('usuarios').doc(uid).set({
41       'nombre': nombre,
42       'email': email,
43       'rol': rol,
44     });
45
46     return result.user; // Retorna el usuario registrado
47   }
48
49   // Stream para escuchar los cambios de autenticación (login/logout en tiempo real)
50   Stream<User?> get userChanges => _auth.authStateChanges();
51 }
```

Las correcciones se muestran en la siguiente imagen

✓ RF- 011 Generacion de Reportes Integrales

Exportar reportes en PDF que incluyan historial de sensores, mantenimientos realizados y observaciones registradas.

```
Constru_Monitoreo-master > monitoreo_apicola > lib > controllers > report_controller.dart
1  import 'package:flutter/material.dart';
2  import '../models/report_model.dart';
3  import '../services/report_service.dart';
4  import '../services/firestore_service.dart';
5  import '../services/pdf_service.dart';
6
7  class ReportController extends ChangeNotifier {
8    final ReportService _reportService = ReportService();
9    final FirestoreService _firestoreService = FirestoreService();
10   final PdfService _pdfService = PdfService();
11
12   bool _isLoading = false;
13   String? _error;
14   List<ReportModel> _reports = [];
15   ReportModel? _currentReport;
16
17   // Getters
18   bool get isLoading => _isLoading;
19   String? get error => _error;
20   List<ReportModel> get reports => _reports;
21   ReportModel? get currentReport => _currentReport;
22
23   // Generar reporte integral
24   Future<String?> generateIntegralReport([
25     required String colmenaId,
26     required DateTime fechaInicio,
27     required DateTime fechaFin,
28     String tipoReporte = 'integral',
29   ]) async {
30     try {
31       _setLoading(true);
32       _clearError();
33
34       // Obtener datos de la colmena
35       final colmenaData = await _firestoreService.getColmenaById(colmenaId);
36       if (colmenaData == null) {
37         throw Exception('Colmena no encontrada');
38       }
39
40       // Obtener historial de sensores
41       final sensores = await _firestoreService.getSensorHistory(
42         colmenaId,
43         fechaInicio,
44         fechaFin,
45       );
46     } catch (e) {
47       _setError(e.toString());
48     }
49   }
50 }
```

En la imagen observamos la falta de comentarios

DESCRIPCIÓN DEL CÓDIGO

El código implementa un controlador de reportes en Flutter, siguiendo el patrón ChangeNotifier para actualizar la interfaz cuando cambien los datos.

Está estructurado en la clase ReportController, que se encarga de:

1. Conexión con servicios
 - FirestoreService para obtener datos de colmenas, sensores, mantenimientos y observaciones.
 - ReportService para guardar, obtener y eliminar reportes en la base de datos.
 - PdfService para generar y compartir reportes en formato PDF.
2. Método generateIntegralReport
 - Recolecta información de una colmena en un rango de fechas, incluyendo datos de sensores, mantenimientos y observaciones.
 - Crea un objeto ReportModel con la información obtenida.
 - Genera un archivo PDF del reporte y lo guarda en Firestore.
 - Actualiza la lista de reportes disponibles en memoria.
3. Método loadReports
 - Recupera reportes almacenados, filtrados opcionalmente por colmenalid.
 - Actualiza la lista local para mostrarlos en la interfaz.
4. Método deleteReport
 - Elimina un reporte de la base de datos y de la lista local.
5. Método shareReport
 - Genera un PDF del reporte seleccionado y lo comparte usando el servicio correspondiente.
6. Gestión de estado
 - Maneja indicadores de carga (isLoading), errores (error) y actualiza la interfaz cuando se realizan cambios (notifyListeners).

Propósito dentro del proyecto

En el contexto del Aplicativo de Monitoreo en Tiempo Real basado en IoT para el Control del Hábitat Apícola:

- ✓ Este módulo centraliza la generación de reportes que integran toda la información relevante de cada colmena.
- ✓ Facilita la exportación en PDF para respaldos y análisis externos.
- ✓ Permite a los usuarios consultar el historial y compartir reportes de forma ágil, mejorando la gestión de datos apícolas.

Mejoras realizadas

```
1  import 'package:flutter/material.dart';
2  import '../models/report_model.dart';
3  import '../services/report_service.dart';
4  import '../services/firestore_service.dart';
5  import '../services/pdf_service.dart';
6
7  /// Controlador para la gestión de reportes.
8  /// 💡 Mejora: Comentarios descriptivos agregados para facilitar la comprensión.
9  class ReportController extends ChangeNotifier {
10     // Servicios utilizados para manejar reportes, datos en Firestore y PDF.
11     // 💡 Mejora: Comentario aclarando la función de cada servicio.
12     final ReportService _reportService = ReportService();
13     final FirestoreService _firestoreService = FirestoreService();
14     final PdfService _pdfService = PdfService();
15
16     // Estado interno del controlador
17     bool _isLoading = false; // Indica si hay una operación en progreso.
18     String? _error; // Mensaje de error actual.
19     List<ReportModel> _reports = []; // Lista de reportes cargados.
20     ReportModel? _currentReport; // Reporte actual en uso.
21
22     // Getters para exponer el estado.
23     bool get isLoading => _isLoading;
24     String? get error => _error;
25     List<ReportModel> get reports => _reports;
26     ReportModel? get currentReport => _currentReport;
27 }
```

Las mejoras realizadas se observan como los comentarios descriptivos la estructura esta clara paso a paso, validacion de datos e incluir los mensajes de error sobre cada accion.

report_model.dart

```
Constru_Monitoreo-master > monitoreo_apicola > lib > models > report_model.dart
1 class ReportModel {
2   final String id;
3   final String colmenaId;
4   final String colmenaNombre;
5   final DateTime fechaGeneracion;
6   final List<SensorData> historialSensores;
7   final List<MantenimientoRecord> mantenimientos;
8   final List<ObservacionRecord> observaciones;
9   final String tipoReporte;
10  final DateTime fechaInicio;
11  final DateTime fechaFin;
12
13  ReportModel({
14    required this.id,
15    required this.colmenaId,
16    required this.colmenaNombre,
17    required this.fechaGeneracion,
18    required this.historialSensores,
19    required this.mantenimientos,
20    required this.observaciones,
21    required this.tipoReporte,
22    required this.fechaInicio,
23    required this.fechaFin,
24  });
25
26  Map<String, dynamic> toJson() {
27    return {
28      'id': id,
29      'colmenaId': colmenaId,
30      'colmenaNombre': colmenaNombre,
31      'fechaGeneracion': fechaGeneracion.toIso8601String(),
32      'historialSensores': historialSensores.map((s) => s.toJson()).toList(),
33      'mantenimientos': mantenimientos.map((m) => m.toJson()).toList(),
34      'observaciones': observaciones.map((o) => o.toJson()).toList(),
35      'tipoReporte': tipoReporte,
36      'fechaInicio': fechaInicio.toIso8601String(),
37      'fechaFin': fechaFin.toIso8601String(),
38    };
39  }
40
41  factory ReportModel.fromJson(Map<String, dynamic> json) {
42    return ReportModel(
43      id: json['id'],
44      colmenaId: json['colmenaId'],
45      colmenaNombre: json['colmenaNombre'],
46      fechaGeneracion: DateTime.parse(json['fechaGeneracion']),
47      historialSensores: (json['historialSensores'] as List)
48        .map((s) => SensorData.fromJson(s))
49        .toList(),
50      mantenimientos: (json['mantenimientos'] as List)
51        .map((m) => MantenimientoRecord.fromJson(m))
52        .toList(),
53      observaciones: (json['observaciones'] as List)
54        .map((o) => ObservacionRecord.fromJson(o))
55        .toList(),
56      tipoReporte: json['tipoReporte'],
57      fechaInicio: DateTime.parse(json['fechaInicio']),
58      fechaFin: DateTime.parse(json['fechaFin']),
59    );
60  }
```

Este código modela la estructura de reportes de colmenas en Flutter/Dart mediante las clases ReportModel, SensorData, MantenimientoRecord y ObservacionRecord, con métodos para convertir datos a/desde JSON. Presenta errores y riesgos como falta de validaciones y manejo de excepciones en fromJson, ausencia de comentarios descriptivos y poca robustez ante datos nulos. Se recomienda agregar validaciones, try-catch en la deserialización y comentarios explicativos para mejorar su claridad y confiabilidad.

report_view.dart

```
Constru_Monitoreo-master > monitoreo_apicola > lib > views > report_view.dart
1  import 'package:flutter/material.dart';
2  import 'package:cloud_firestore/cloud_firestore.dart';
3  import 'package:firebase_auth/firebase_auth.dart';
4  import 'package:pdf/pdf.dart';
5  import 'package:pdf/widgets.dart' as pw;
6  import 'package:path_provider/path_provider.dart';
7  import 'package:share_plus/share_plus.dart';
8  import 'dart:io';
9  import 'package:intl/intl.dart';
10
11 class SimpleReportView extends StatefulWidget {
12   const SimpleReportView({super.key});
13
14   @override
15   State<SimpleReportView> createState() => _SimpleReportViewState();
16 }
17
18 class _SimpleReportViewState extends State<SimpleReportView> {
19   bool _isGenerating = false;
20   Map<String, int> contadores = {
21     'colmenas': 0,
22     'mantenimientos': 0,
23     'observaciones': 0,
24     'usuarios': 0,
25     'lecturas': 0,
26     'umbrales': 0,
27     'reportes': 0,
28   };
29
30   Map<String, dynamic> datosSensores = {
31     'co2Promedio': 0.0,
32     'sonidoPromedio': 0.0,
33     'co2Maximo': 0.0,
34     'sonidoMaximo': 0.0,
35     'co2Minimo': 0.0,
36     'sonidoMinimo': 0.0,
37     'lecturasTotales': 0,
38     'lecturasAlerta': 0,
39     'ultimaLectura': null,
40   };
41
42   @override
43   void initState() {
44     super.initState();
45     _cargarContadores();
46   }
```

Este código define modelos de datos para reportes de colmenas en Flutter/Dart (*ReportModel*, *SensorData*, *MantenimientoRecord*, *ObservacionRecord*), con funciones de serialización y deserialización a JSON. Entre sus errores y áreas de mejora están: falta de validaciones y manejo de excepciones en *fromJson*, ausencia de comprobaciones ante valores nulos o tipos inválidos, y carencia de comentarios que faciliten su mantenimiento. Se recomienda implementar validaciones, *try-catch* para la carga de datos y documentación interna para aumentar la robustez y legibilidad.

✓ RF-003 Gestión de Colmena

Registrar y actualizar información de una única colmena (ubicación, estado y datos técnicos).

colmena_model.dart

1. Deserialización segura

Descripción: Antes se asumía que los campos de Firestore existían y eran String. Ahora se convierten siempre a string, se limpian espacios y se toleran valores null.

```
factory Colmena.fromMap(Map<String, dynamic> data, String documentId) {  
  return Colmena(  
    id: documentId,  
    ubicacion: (data['ubicacion'] ?? '').toString().trim(),  
    estado: (data['estado'] ?? '').toString().trim(),  
    descripcionTecnica: (data['descripcionTecnica'] ?? '').toString().trim(),  
  );  
}
```

Mejora: Evita errores por null o tipos distintos a String y asegura datos limpios.

2. Sanitización en la escritura

Descripción: Antes los valores se guardaban tal cual, incluso con espacios extra. Ahora se limpian antes de enviarlos a Firestore.

```
Map<String, dynamic> toMap() {  
  return {  
    'ubicacion': ubicacion.trim(),  
    'estado': estado.trim(),  
    'descripcionTecnica': descripcionTecnica.trim(),  
  };  
}
```

Mejora: Garantiza datos consistentes para búsquedas y comparaciones.

3. Validación interna

Descripción: No existía un método para verificar si un objeto estaba listo para guardarse. Ahora hay una validación centralizada.

```
bool esValida() {  
  return ubicacion.isNotEmpty &&  
    estado.isNotEmpty &&  
    descripcionTecnica.isNotEmpty;  
}
```

Mejora: Previene registros incompletos o inválidos en la base de datos.

4. Actualización parcial

Descripción: Antes había que crear un objeto nuevo con todos los datos, aunque solo cambiara uno. Ahora se puede copiar y modificar solo lo necesario.

```
Colmena copiarCon({
  String? ubicacion,
  String? estado,
  String? descripcionTecnica,
}) {
  return Colmena(
    id: id, // el ID no cambia aquí
    ubicacion: (ubicacion ?? this.ubicacion).trim(),
    estado: (estado ?? this.estado).trim(),
    descripcionTecnica:
      (descripcionTecnica ?? this.descripcionTecnica).trim(),
  );
}
```

Mejora: Facilita actualizaciones sin riesgo de perder datos existentes.

colmena_controller.dart

1. Descripción:

Maneja **lista** de colmenas (no cumple RF-003: **una sola colmena**).

Sin **validación** (esValida()), guarda datos incompletos.

Sin **manejo de errores** ni logs.

No hay **stream** para tiempo real.

Posible **duplicidad** al usar add().

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/colmena_model.dart';

class ColmenaController {
  final _ref = FirebaseFirestore.instance.collection('colmenas');

  Future<List<Colmena>> getColmenas() async {
    final snapshot = await _ref.get();
    return snapshot.docs
      .map((doc) => Colmena.fromMap(doc.data(), doc.id))
      .toList();
  }

  Future<void> addColmena(Colmena colmena) async {
    await _ref.add(colmena.toMap());
  }

  Future<void> updateColmena(Colmena colmena) async {
    await _ref.doc(colmena.id).update(colmena.toMap());
  }

  Future<void> deleteColmena(String id) async {
    await _ref.doc(id).delete();
  }
}
```

Mejoras:

- **Lista → único registro:** se sustituyen operaciones masivas por limit(1) y métodos enfocados a **una colmena**.
- **Upsert con merge:** registrarOActualizarColmena() evita duplicados y actualiza solo campos cambiados.
- **Validación previa:** se usa esValida() para impedir registros incompletos.
- **Tiempo real:** escucharColmenaUnica() listo para StreamBuilder.
- **Errores controlados:** try/catch + rethrow para que la UI pueda mostrar mensajes claros.

colmena_view.dart

1. Implementación del método build()

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("Gestión de Colmena"),
      backgroundColor: Colors.teal,
    ), // AppBar
```

- *Antes:* La clase State no implementaba el método build(), provocando el error "Missing concrete implementation of abstract method build".
- *Después:* Se agregó el método build() con un Scaffold, AppBar, y ListView.builder para mostrar el listado de colmenas o un CircularProgressIndicator mientras carga.

2. Reemplazo del método inexistente eliminarColmenaU_

```
/// Elimina una colmena por su ID.
Future<void> _eliminarColmena(String id) async {
  try {
    await _colmenaController.deleteColmena(id);
    await _cargarColmenas();
  } catch (e) {
    debugPrint("Error al eliminar colmena: $e");
  }
}
```

- *Antes:* Se intentaba llamar a _colmenaController.eliminarColmenaU_(), un método que no existía ni en el modelo ni en el controlador.
- *Después:* Se reemplazó por _eliminarColmena(id) que internamente usa deleteColmena(id) del ColmenaController.

3. Corrección de try sin catch

```
try {
  await _colmenaController.updateColmena(colmena);
  await _cargarColmenas();
} catch (e) {
  debugPrint("Error al actualizar colmena: $e");
}
```

- *Antes:* El bloque try no tenía catch ni finally, generando error de sintaxis.
- *Después:* Se añadió catch (e) para capturar errores y mostrar mensajes con debugPrint.

4. Actualización del botón de agregar colmena

```
/// Abre el formulario para crear una nueva colmena.  
void _abrirFormularioNueva() {  
  _ubicacionController.clear();  
  _estadoController.clear();  
  _descripcionController.clear();  
}
```

- *Antes:* El botón llamaba a un método vacío _abrirFormulario.
- *Después:* Ahora llama a _abrirFormularioNueva() que limpia campos y abre un AlertDialog para registrar colmenas en Firestore.

2. Conclusiones

El problema principal se debe a la ausencia del paquete cmdline-tools, que es esencial para que Flutter gestione licencias y compile para Android.

Sin aceptar las licencias del SDK (flutter doctor --android-licenses) no podrás compilar ni ejecutar apps en dispositivos Android, incluso si tienes Android Studio instalado.

La solución pasa por instalar cmdline-tools y configurar correctamente la variable ANDROID_SDK_ROOT, lo que permitirá que Flutter reconozca el SDK y elimine el error en la Android toolchain.